1 Trabajo Práctico 2 – Sistemas de Computación

Integrantes:

- Brigido Tomás
- Siñuka Javier
- Tosolini Agustín

1 **Trabajo Práctico 2 – Sistemas de Computación** **1**

1.1 Introducción 3

1.2 Descripción General del Proyecto 4

1.3 Tecnologías Utilizadas 5

1.4 Variante without_docker 6

1.4.1 Flujo de trabajo 6

1.4.2 Componentes principales 6

1.5 Variante docker_test 7

1.5.1 Archivo api.py 7

1.5.2 Obtención de datos 7

1.5.3 Modelo de datos 8

1.5.4 Biblioteca compartida y ctypes 8

1.5.5 Transformación C y Assembly 8

1.5.6 Postprocesamiento y visualización 9

1.5.7 Endpoint HTTP 9

1.6 Automatización con Docker 10

1.6.1 Dockerfile 10

1.6.2 Script de ejecución 10

1.6.3 Docker Compose 10

1.7 Flujo Completo del Sistema 12

1.8 Aspectos Relevantes de Sistemas de Computación 13

1.9 Conclusión 14

1.1 Introducción

El presente trabajo práctico se centra en el estudio y la verificación de las convenciones de llamada entre distintos lenguajes y niveles de abstracción. El objetivo principal del proyecto no es solamente procesar un conjunto de datos, sino comprobar en la práctica cómo deben intercambiarse argumentos, valores de retorno y estructuras de memoria cuando intervienen componentes escritos en Python, C y lenguaje ensamblador.

Para llevar adelante esta comprobación, el proyecto utiliza un conjunto de herramientas que permiten poner a prueba dicha convención de llamada en un escenario real. Python se emplea como lenguaje de orquestación y como punto de entrada del sistema; C funciona como capa intermedia para definir una interfaz estable con la rutina nativa; y Assembly implementa la lógica de conversión numérica. A su vez, el uso de `ctypes`, bibliotecas compartidas, scripts de compilación y contenedores Docker permite verificar que la comunicación entre capas se produzca correctamente y de forma reproducible.

Con el fin de contar con datos reales sobre los cuales ejecutar estas pruebas, se toma como fuente la API pública del Banco Mundial y se trabaja con valores del índice de Gini. De este modo, el proyecto combina un caso concreto de procesamiento de datos con un objetivo técnico más profundo: validar que las convenciones de llamada se respeten correctamente a lo largo de todo el flujo, desde la obtención de la información hasta la ejecución del código nativo y la recuperación de los resultados en Python.

1.2 Descripción General del Proyecto

En este proyecto se pueden encontrar dos variantes del trabajo. La primera, `without_docker`, representa una aproximación inicial en la que Python obtiene los datos y delega la conversión a una biblioteca de 32 bits utilizando `msl.loadlib`. La segunda, `docker_test`, constituye una versión más integrada del flujo completo, donde la compilación del código nativo y la ejecución del pipeline se realizan dentro de un contenedor Docker.

Desde el punto de vista funcional, ambas variantes comparten la misma idea general: partir de un conjunto de valores del índice de Gini, convertirlos de `float` a `int` mediante código nativo y presentar el resultado final de forma procesable desde Python. La versión con Docker amplía este flujo con la generación de gráficos y con una API HTTP sencilla construida con Flask.

1.3 Tecnologías Utilizadas

El proyecto combina herramientas de distinto nivel, cada una con una responsabilidad bien definida:

Tecnología	**Rol dentro del proyecto**
Python 3	Orquestación general, consumo de API, generación de gráficos y exposición HTTP.
C	Capa puente entre Python y la rutina de bajo nivel.
NASM Assembly	Conversión numérica de punto flotante a entero.
Flask	Exposición de un endpoint simple para devolver datos procesables.
Matplotlib	Generación del gráfico final en formato PNG.
Docker / Docker Compose	Reproducibilidad del entorno de compilación y ejecución.
Banco Mundial API	Fuente externa de datos del índice de Gini.

1.1

La elección de estas herramientas refleja un enfoque de integración multicapa. Python resuelve la lógica de alto nivel con rapidez de desarrollo, mientras que C y Assembly se utilizan para abordar la transformación numérica desde un enfoque más cercano al hardware y a las convenciones de ejecución.

1.4 Variante `without_docker`

La variante `without_docker` funciona como una primera implementación del problema. En esta versión, el archivo `main.py` consulta la API del Banco Mundial para obtener los valores del indicador `SI.POV.GINI` entre 2011 y 2020 y luego usa un cliente de `msl.loadlib` para enviar la lista de flotantes hacia un proceso de 32 bits.

1.4.1 Flujo de trabajo

El flujo de esta variante puede describirse de la siguiente manera:

1. Python consulta la API del Banco Mundial.
2. Se filtran registros por código ISO del país.
3. El cliente `Client64` invoca al servidor `Server32`.
4. El servidor arma arreglos de `ctypes` en memoria de 32 bits.
5. La biblioteca `libbridge.so` procesa cada valor usando C y Assembly.
6. Los resultados enteros vuelven al proceso Python como una lista.

1.4.2 Componentes principales

`main.py` implementa la consulta externa, el filtrado y la prueba de extremo a extremo. La función `obtener_gini_pais` construye una URL de la API, descarga la respuesta JSON y extrae los valores válidos para el país solicitado.

`client.py` define la clase `ClienteBridge`, que hereda de `Client64` y utiliza el método `request32` para comunicarse con el servidor de 32 bits.

`server.py` define la clase `Server`, heredada de `Server32`. Allí se carga la biblioteca compartida, se especifica la firma de la función nativa y se transforman las listas Python en arreglos de `ctypes`.

`bridge.c` contiene la función `bridge`, encargada de recorrer un arreglo de `float` y aplicar la función `convert` sobre cada elemento, dejando el resultado en un arreglo de enteros.

`convert.asm` implementa la conversión de un `float` a `int` mediante la FPU, y posteriormente incrementa el resultado en una unidad. Esto significa que en esta primera versión la lógica de `+1` forma parte del código Assembly.

1.5 Variante `docker_test`

La variante `docker_test` representa la versión más madura del proyecto. Aquí se encuentra un pipeline completo donde Python obtiene los datos, delega la conversión a una biblioteca nativa compilada dentro del contenedor, ajusta los resultados y genera una visualización final.

1.5.1 Archivo `api.py`

El archivo `api.py` concentra la mayor parte de la lógica del sistema. Su estructura interna puede separarse en varios bloques funcionales:

Bloque	**Función principal**
Configuración	Define indicador, país, años, rutas y timeout.
Acceso a API	Construye URLs, realiza la consulta y obtiene registros JSON.
Filtrado	Selecciona registros válidos para un país determinado.
Interoperabilidad nativa	Carga la biblioteca compartida y define `argtypes` y `restype`.
Transformación	Convierte arreglos de `float` a `int` mediante C/ASM.
Postprocesamiento	Incrementa los valores convertidos en `+1`.
Visualización	Genera y guarda un gráfico con Matplotlib.
Exposición HTTP	Devuelve datos de Argentina en un endpoint Flask.

1.2

Esta centralización de responsabilidades facilita seguir el flujo completo del pipeline, aunque también implica que gran parte de la lógica está reunida en un mismo archivo. Para un proyecto académico esto resulta aceptable, ya que permite comprender de manera directa la secuencia de pasos involucrados.

1.5.2 Obtención de datos

La URL base utilizada responde a la forma:

`https://api.worldbank.org/v2/en/country/{country_scope}/indicator/{indicator}`

El indicador seleccionado es `SI.POV.GINI`, que corresponde al índice de Gini. Los parámetros construidos incluyen:

- `format=json`
- `date=2011:2020`
- `page=1`
- `per_page=32500`

La cantidad elevada de elementos por página busca evitar problemas de paginación y asegurar que todos los registros relevantes sean descargados en una sola consulta. Una vez obtenida la respuesta, el programa verifica que el payload tenga la estructura esperada y luego recorre los registros para quedarse únicamente con aquellos que pertenecen al país solicitado y poseen un valor numérico no nulo.

1.5.3 Modelo de datos

La clase `IndicatorPoint`, declarada como `dataclass`, encapsula dos atributos: el año y el valor del indicador. Esta decisión mejora la legibilidad del código porque evita manipular directamente diccionarios crudos a lo largo de todo el pipeline y permite ordenar los resultados de forma clara por año.

1.5.4 Biblioteca compartida y `ctypes`

Para la interoperabilidad con código nativo, `api.py` utiliza `ctypes.CDLL` para cargar el archivo `libfloat_to_int.so`. Una vez abierta la biblioteca, se especifica la firma de la función exportada `convert_float_array_to_int(const float*, int*, int)`.

El uso de `argtypes` y `restype` es importante porque le indica a Python cómo debe pasar los argumentos a la función nativa y cómo interpretar su valor de retorno. Esto reduce errores de interoperabilidad y deja explícito el contrato entre ambos mundos.

1.5.5 Transformación C y Assembly

El archivo `interface_w_asm.c` implementa la función intermedia `convert_float_array_to_int`. Su responsabilidad principal es validar que los punteros de entrada y salida no sean nulos y que la cantidad de elementos no sea negativa. Si la

validación es correcta, delega el trabajo real a la rutina Assembly `asm_convert_float_array_to_int`.

Por su parte, `float_to_int.asm` implementa la lógica de recorrido del arreglo. La rutina recibe:

- `rdi`: puntero al arreglo de `float`
- `rsi`: puntero al arreglo de salida `int`
- `edx`: cantidad de elementos

Luego, para cada posición:

1. Carga el valor flotante en `xmm0` mediante `movss`.
2. Convierte el escalar a entero con `cvttss2si`.
3. Guarda el resultado en el arreglo de salida.
4. Incrementa el índice y repite hasta completar el arreglo.

La instrucción `cvttss2si` realiza una conversión con truncamiento hacia cero. Esto significa que los decimales se descartan, y que la suma posterior de `+1` no se realiza en Assembly en esta versión, sino en Python mediante la función `increment_values`.

1.5.6 Postprocesamiento y visualización

Una vez obtenidos los enteros convertidos, el programa aplica una corrección simple sumando una unidad a cada valor. Esta decisión desacopla la conversión numérica de la regla de negocio adicional, dejando más claro qué parte del pipeline corresponde a bajo nivel y cuál pertenece a la lógica de aplicación.

Después del ajuste, se genera un gráfico con Matplotlib. El archivo de salida se guarda dentro de `runtime/logs` con un nombre que incluye fecha y hora. Esto permite conservar evidencia de distintas ejecuciones y comparar resultados a lo largo del tiempo.

1.5.7 Endpoint HTTP

Además del pipeline principal, `api.py` declara un endpoint Flask en `/api/gini/argentina`. Este endpoint devuelve un arreglo JSON con los valores del indicador correspondientes a Argentina, sin aplicar necesariamente toda la etapa de transformación ni la generación del gráfico. Su inclusión es útil como extensión del trabajo, ya que muestra cómo el proyecto también puede exponer datos a otros consumidores.

1.6 Automatización con Docker

La contenerización del proyecto está resuelta mediante `Dockerfile`, `docker-compose.yml` y `run_pipeline.sh`.

1.6.1 Dockerfile

El `Dockerfile` parte de la imagen `python:3.12-slim`, instala `build-essential` y `nasm`, copia el proyecto al directorio de trabajo y luego instala las dependencias Python definidas en `requirements.txt`. Estas dependencias son:

Paquete	**Versión**
Flask	3.1.0
matplotlib	3.10.0

1.3

Esta imagen resulta adecuada para el trabajo porque incorpora un entorno Python moderno y agrega únicamente las herramientas necesarias para compilar C y Assembly.

1.6.2 Script de ejecución

El archivo `run_pipeline.sh` automatiza el proceso completo:

1. Crea directorios de salida para compilación y logs.
2. Compila `float_to_int.asm` con NASM a un objeto ELF64.
3. Genera la biblioteca compartida `libfloat_to_int.so` con GCC.
4. Ejecuta `api.py` con Python 3.

Con esta secuencia, cualquier usuario que construya el contenedor puede reproducir el pipeline sin realizar configuraciones manuales en su sistema anfitrión.

1.6.3 Docker Compose

El archivo `docker-compose.yml` define el servicio `gini_pipeline`, monta el proyecto dentro de `/workspace`, fija variables de entorno para las rutas de build y logs, y ejecuta el script

principal. Esta capa agrega comodidad para la puesta en marcha y hace más clara la configuración de ejecución del trabajo.

1.7 Flujo Completo del Sistema

La lógica integral del proyecto, tomando como referencia principal la variante `docker_test`, puede resumirse con el siguiente esquema:

1. Python construye la consulta a la API del Banco Mundial.
2. Se descargan los registros del indicador GINI.
3. Se filtran los datos correspondientes al país elegido.
4. Los valores se transforman en un arreglo de `float` manejable por `ctypes`.
5. Python invoca una función exportada por una biblioteca compartida.
6. C valida los argumentos y llama a la rutina Assembly.
7. Assembly convierte cada valor `float` a `int`.
8. Python recibe el arreglo convertido.
9. Se suma `+1` a cada resultado.
10. Se genera un gráfico PNG y se almacenan los resultados.

Este encadenamiento muestra con claridad cómo un mismo problema puede resolverse distribuyendo responsabilidades entre distintos niveles tecnológicos. Lejos de ser una implementación puramente académica, el trabajo refleja un patrón real de integración entre lógica de aplicación y componentes nativos.

1.8 Aspectos Relevantes de Sistemas de Computación

Desde la perspectiva de la materia, este trabajo permite discutir varios temas importantes:

Concepto	**Aplicación en el proyecto**
Convención de llamada	El código nativo debe respetar la ABI esperada por C y Python.
Representación de datos	Los valores pasan de JSON a `float`, luego a `int` y finalmente a imagen o JSON.
Interoperabilidad	Se enlazan componentes escritos en distintos lenguajes.
Memoria	Se manipulan punteros, arreglos contiguos y tipos primitivos.
Portabilidad	Docker encapsula herramientas y dependencias del entorno.
Modularidad	El pipeline se divide en etapas con responsabilidades específicas.

1.4

En particular, la interoperabilidad entre Python, C y Assembly obliga a tener en cuenta detalles que en un programa puramente de alto nivel suelen quedar ocultos. Tipos de datos, tamaño de palabra, paso de argumentos, formato del objeto compilado y convenciones de ejecución se vuelven aspectos explícitos del diseño y de la depuración.

1.9 Conclusión

En este trabajo se desarrolla una integración entre distintos niveles de programación. A partir de un problema concreto, como la obtención y transformación del índice de Gini, el proyecto combina acceso a servicios externos, procesamiento de datos, interoperabilidad con bibliotecas nativas, programación en C y rutinas en Assembly.

En términos de Sistemas de Computación, el trabajo resulta especialmente interesante porque obliga a articular conceptos de alto y bajo nivel dentro de una misma aplicación. El uso de `ctypes`, bibliotecas compartidas, punteros, convenciones de llamada, compilación y automatización con Docker convierte a este proyecto en un ejercicio representativo de integración entre software de aplicación y componentes cercanos a la arquitectura de la máquina.